

Firebase Push Notifications — Flutter Sample App

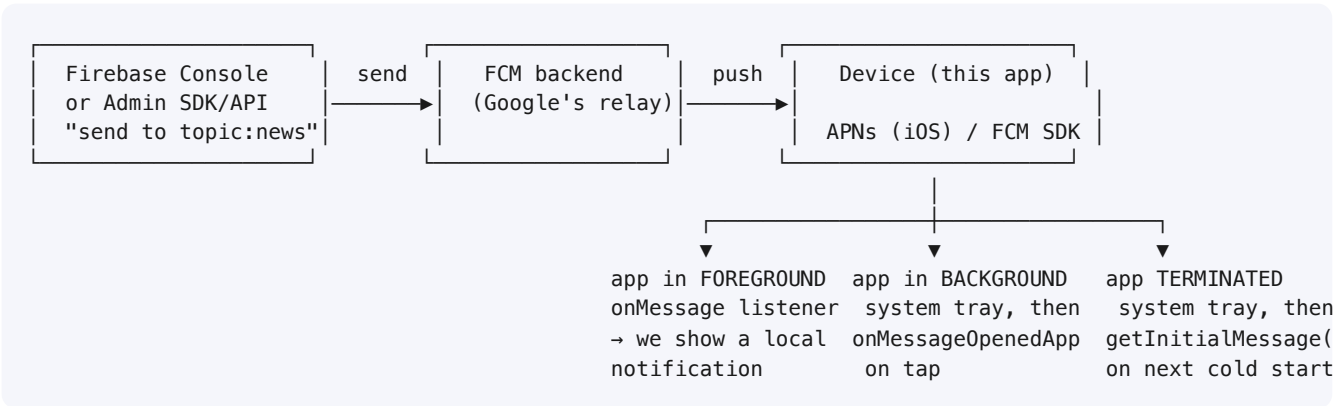
How the app works, how it's configured, and how to test it end to end

Project: pushnotification • Firebase project: pushnotification-4281b • Generated 2026-08-06

1. What this app is

A minimal Flutter app that demonstrates Firebase Cloud Messaging (FCM) end to end: requesting notification permission, obtaining a device token, subscribing to a topic, and receiving messages in all three app lifecycle states — foreground, background, and terminated.

2. How Firebase Cloud Messaging works (the big picture)



FCM never shows a notification banner while the app is in the foreground — that's why the app uses `flutter_local_notifications` to display one manually when `onMessage` fires. In background/terminated states, the OS shows the system notification automatically from the payload.

3. Project structure — what was added

Path	Purpose
<code>lib/main.dart</code>	App entry point + FCM wiring (permissions, token, listeners, UI)
<code>lib/firebase_options.dart</code>	Auto-generated by <code>flutterfire configure</code> ; per-platform Firebase config values
<code>android/app/google-services.json</code>	Android app registration data from your Firebase project
<code>ios/Runner/GoogleService-Info.plist</code>	iOS app registration data from your Firebase project
<code>android/app/src/main/AndroidManifest.xml</code>	Added <code>POST_NOTIFICATIONS</code> permission + default notification channel
<code>android/app/build.gradle.kts</code>	Enabled core library desugaring (required by local notifications plugin)
<code>ios/Runner/Info.plist</code>	Added <code>UIBackgroundModes: remote-notification</code>

```
ios/Runner.xcodeproj/project.pbxproj
```

Raised iOS deployment target 13.0 → 15.0 (required by firebase_core 4.x)

4. What happens when the app starts (`lib/main.dart` walkthrough)

4.1 Initialization

```
await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
FirebaseMessaging.onBackgroundMessage(firebaseMessagingBackgroundHandler);
await _initLocalNotifications();
```

Firebase is initialized with the config generated for the current platform. The background handler is registered *before* `runApp()` — it must be a top-level function because Android/iOS run it in a separate isolate when a message arrives while the app isn't running.

4.2 Requesting permission & getting the token

```
final settings = await _messaging.requestPermission(alert: true, badge: true, sound: true);
final token = await _messaging.getToken();
```

The token uniquely identifies this app install on this device. It's what you'd send to your own backend to target a specific user, and it's what the Firebase Console's "test message" feature uses. It's shown in the app UI with a copy button.

4.3 Listening for messages

Listener	Fires when	What the app does
<code>FirebaseMessaging.onMessage</code>	Message arrives while app is open (foreground)	Logs it in the list + shows a local notification
<code>FirebaseMessaging.onMessageOpenedApp</code>	User taps a notification that arrived while backgrounded	Logs it in the list
<code>_messaging.getInitialMessage()</code>	App is cold-started by tapping a notification	Logs it in the list
<code>firebaseMessagingBackgroundHandler</code>	Message arrives, app backgrounded/terminated	Runs in a separate isolate; just re-initiates Firebase (OS shows the notification itself)

4.4 Topics

Topics let you broadcast to every subscribed device without tracking individual tokens. Tapping **Subscribe** in the app calls `_messaging.subscribeToTopic('news')` (or whatever you typed in the Topic field). **Unsubscribe** calls the mirror method.

5. Platform-specific setup explained

5.1 Android

- **POST_NOTIFICATIONS permission** — required on Android 13+ or notifications are silently blocked.
- **Notification channel** (`high_importance_channel`) — Android 8+ requires every notification to belong to a channel; created at startup and referenced in the manifest as the default FCM channel.
- **Core library desugaring** — `flutter_local_notifications` uses Java 8+ APIs that need desugaring support on older Android API levels; enabled in `build.gradle.kts` .

5.2 iOS

- **UIBackgroundModes: remote-notification** — lets the app wake briefly to process a push while backgrounded.
- **Deployment target 15.0** — required by the version of `firebase_core` used here.
- **Push Notifications capability (not yet done)** — see the "What's still needed" section below.

6. How to test the app

6.1 Run it

```
flutter run
```

Grant the notification permission prompt, then note or copy the device token shown on screen.

6.2 Subscribe to a topic

Type a topic name (default `news`) and tap **Subscribe**. The button changes to "Unsubscribe" once it succeeds.

6.3 Send a test notification — by topic

1. Open the [Firebase Console → Messaging](#).
2. **New campaign** → **Notifications**.
3. Enter a title and body.
4. Under **Target**, choose **Topic** and enter the exact same topic name used in the app.
5. **Review** → **Publish**.

6.4 Send a test notification — by device token

In the same compose screen, use the **"Send test message"** button and paste the token copied from the app. This targets one specific device/install regardless of topic subscription — useful for quick single-device checks.

Expected results

- App open (foreground): notification appears in the app's own "Received messages" list, plus a system-style local notification.
- App backgrounded: standard system notification appears; tapping it opens the app and logs "opened from background".
- App killed: same as above, logged as "opened from terminated" on next launch.

7. Issues hit during setup, and the fixes

Problem	Cause	Fix
<code>flutterfire configure</code> crashed with a Ruby <code>xcodeproj</code> LoadError	macOS's bundled system Ruby (2.6) can't build the native gem extension needed by the CLI's Xcode-project editing step	Installed a modern Ruby via Homebrew and the <code>xcodeproj</code> gem under it, then re-ran configure with that Ruby first on PATH
<code>flutterfire configure</code> initially failed with "Firebase CLI not found"	The FlutterFire CLI shells out to the official Firebase CLI, which wasn't installed	Installed it via <code>brew install firebase-cli</code>
Android build failed: "flutter_local_notifications requires core library desugaring"	The plugin depends on desugared Java 8+ APIs not available by default on older Android targets	Enabled <code>isCoreLibraryDesugaringEnabled = true</code> and added the <code>desugar_jdk_libs</code> dependency
Android build failed with a <code>jlink</code> error inside <code>core-for-system-modules.jar</code>	The system's default JDK (Zulu 26) is too new / incompatible with the Android Gradle Plugin's JDK-image transform	Pointed Flutter at Android Studio's bundled JDK 21 via <code>flutter config --jdk-dir=...</code> (persists for all future builds on this machine)
One build failed on <code>stripDebugDebugSymbols</code>	Two Gradle builds ran concurrently and raced on the same output directory (not a real bug)	Re-ran cleanly; not seen again

8. What's still needed from you

iOS Push Notifications capability — the simulator log shows `"no valid aps-environment entitlement string found for application"`. Real push delivery on a physical iPhone requires enabling the **Push Notifications** capability in Xcode, which needs your own Apple Developer account/team. This can't be done without your login, so it wasn't automated.

To fix: open `ios/Runner.xcworkspace` in Xcode → select the *Runner* target → *Signing & Capabilities* → select your Team → + *Capability* → *Push Notifications*. Android needs no equivalent step.

9. Quick reference — key commands

```
# Run the app on a connected device/simulator
flutter run

# Build a debug APK
flutter build apk --debug

# Build for the iOS simulator
flutter build ios --simulator --debug

# Regenerate firebase_options.dart if you add/remove platforms
dart pub global run flutterfire_cli:flutterfire configure
```

Generated for the pushnotification Flutter sample app. Firebase project: pushnotification-4281b.